

dashboard_design

Helix OTA — 1.0.0-MVP React Dashboard Design Specification

Field	Value
Revision	1
Created	2026-06-08
Last modified	2026-06-08
Status	active
Status summary	Full design specification of the Helix OTA 1.0.0-MVP operator dashboard: a React single-page application that consumes the <code>/api/v1</code> REST control plane (../api/endpoints.md). Covers secure login (OAuth2 ROPC → JWT access token + rotating refresh token), artifact upload (multipart/form-data with the S1–S6 validation-feedback chain), release/deployment management (all-targets for MVP), and fleet-health + telemetry views. Specifies the component architecture reusing the *-React catalogue bricks, the route map, the auth-context/state model, and a screen→endpoint API-client mapping. Plans the NEW public dashboard repository (GitHub + GitLab) per locked decision D4. Closes gap G6 in ../.. /research/ additions_synthesis.md §8 / §9.1 — the single largest missing document.
Issues	The three *-React submodules (UI-Components-React, Dashboard-Analytics-React, Auth-Context-React) are named in the master design §10 / D7 but are NOT on the verified catalogue (../.. /00-master/ submodule_reuse_map.md §7 explicitly omits them); their existence, ownership, and public API are

Field	Value
	UNVERIFIED (see §13). HelixConstitution clause numbers (§11.4.6, §11.4.4, §11.4.61, §11.4.74, §11.4.28, §7.1) are carried from corpus convention and are UNVERIFIED against the authoritative Constitution text. The artifact-download path Range/HTTP-3 semantics consumed indirectly here are UNVERIFIED pending ADR-0004 §6. No dashboard code exists yet; this is a design, not an as-built.
Fixed	N/A (initial revision).
Continuation	Inspect the three *–React bricks’ real public surface and remove the UNVERIFIED tags or substitute a verified component library (§13.1); create the PUBLIC ota–dashboard repo on GitHub + GitLab and mirror to GitFlic/GitVerse (§11); pin the framework/build toolchain (Vite vs Next; §3.2) once the brick toolchain is confirmed; bind concrete telemetry-read endpoints once gap G4 lands (telemetry read API is PARTIAL in MVP — §9.3); add the four-layer test harness (§12).
Owner	Helix OTA dashboard team
Related	../api/endpoints.md; ../api/openapi.yaml; ../00-master/submodule_reuse_map.md; ../00-master/2026-06-07-helix-ota-design.md; ../research/additions_synthesis.md; ../security/signing_verification.md

table_of_contents

1. [purpose_and_scope](#)
2. [conventions](#)
3. [technology_stack_and_build](#)
4. [component_architecture](#)
5. [catalogue_brick_reuse](#)
6. [route_map](#)
7. [state_and_auth_context](#)
8. [api_client_and_screen_endpoint_mapping](#)
9. [screen_specifications](#)
 - [9.1_login_screen](#)
 - [9.2_artifact_upload_screen](#)

- 9.3 release_and_deployment_management
 - 9.4 fleet_health_and_telemetry_views
 - 10. security_considerations
 - 11. new_public_repository_plan
 - 12. testing_four_layer
 - 13. anti_bluff_unverified_register
 - 14. compliance_notes_helixconstitution
-

1. purpose_and_scope

This document specifies the design of the Helix OTA **1.0.0-MVP** operator dashboard: a browser-based single-page application (SPA) that lets an authenticated operator **log in, upload and validate OTA artifacts, publish releases, create all-targets deployments, and observe fleet health + telemetry**. It is the human front end to the REST control plane defined in `../api/endpoints.md`; it owns **no** OTA business logic of its own — every state-changing action is a call to a documented `/api/v1` endpoint.

The dashboard is mandated by locked decision **D7** (“React dashboard”) and the master architecture §4 (the Dashboard (React) plane that → API). It closes consolidated gap **G6** (“React dashboard (upload→deploy→fleet/telemetry view) ... MISSING — no spec, repo, or code anywhere. Largest single doc gap.”) and the routed work item §9.1 in `../research/additions_synthesis.md`.

In scope (MVP):

- Secure login — OAuth2 Resource Owner Password Credentials → JWT access token + opaque rotating refresh token (`endpoints.md` §4.1, §7).
- Artifact upload — multipart/form-data with file + metadata parts and the S1–S6 validation-feedback chain surfaced to the operator (`endpoints.md` §9.1).
- Release management — create/list/read releases (`endpoints.md` §10).
- Deployment management — create/read **all-targets** deployments (`endpoints.md` §11); staged/percentage rollout UI is **deferred to 1.0.1** with the engine (G7).
- Device registry views — device list + per-device status (`endpoints.md` §8.2).
- Fleet-health + telemetry views — deployment progress aggregates (`endpoints.md` §11.2) and device status; **richer telemetry read/aggregation is PARTIAL** (gap G4) and the dashboard degrades gracefully where those endpoints are not yet built (§9.4).

Out of scope (MVP): staged-rollout controls (1.0.1, G7); device-group CRUD UI (1.0.1, G5); audit-log viewer (1.0.1, G3); user/RBAC administration UI; end-user/multi-version rollback UI (1.0.2+); any non-Android fleet view. These are noted at their screens so the layout reserves space without a breaking redesign.

2. conventions

- **Headings:** lowercase_snake_case per the gap-G6 spec convention requested for this document.
- **Endpoint references** are written `METHOD /api/v1/<path>` and always trace to a named section in `../api/endpoints.md`.

- **Brick references** name the catalogue/*-React submodule and tag **UNVERIFIED** where the public surface was not inspected in this revision (§13).
- **Identifiers** (`release_id`, `deployment_id`, `device_id`, `artifact_id`) are treated as **opaque server-issued strings** (`endpoints.md` §2); the dashboard never parses or mints them.
- **Timestamps** are RFC 3339 UTC strings, rendered to the operator's locale at the edge.
- **S1–S6** denotes the artifact validation-feedback chain (defined in §9.2); it maps onto the server upload checks and error codes in `endpoints.md` §9.1.

3. technology_stack_and_build

3.1 framework

- **React SPA** (mandated by D7 / master §4). TypeScript throughout for type-safe binding to the ota-protocol wire types (the dashboard mirrors the protocol DTOs as generated/ hand-kept TypeScript interfaces; the OpenAPI contract `./api/openapi.yaml` is the source of truth and SHOULD drive client-type generation).
- **Routing**: declarative client-side router with nested protected routes (§6).
- **Server-state / data fetching**: a query-cache layer (request dedup, cache, retry, background refresh) wraps the API client (§8) so telemetry/fleet views can poll without bespoke effect code.
- **Forms + validation**: schema-validated forms for login (§9.1) and upload metadata (§9.2), mirroring the server's validation so obvious errors are caught client-side before the request (the **server remains authoritative** — client validation is UX only).

3.2 build and toolchain

The concrete build toolchain (e.g. **Vite** SPA vs a **Next.js** static-export SPA) is **deferred until the *-React bricks' toolchain is inspected** (§13): the dashboard MUST adopt whatever the reused bricks already standardize on rather than introduce a divergent build (catalogue-first, §11.4.74). Hard requirements regardless of toolchain:

- Static, CDN-deployable build artifacts (no SSR data dependency on secrets); the SPA talks only to `/api/v1` at runtime.
- Runtime config (API base URL, feature flags for deferred screens) injected at deploy time, not hard-coded — mirrors the server config brick philosophy.
- Runs on the containers substrate (master §3 / §11.4.76) for the dev stack and CI build.

UNVERIFIED: the bricks' actual framework/bundler choice; do not assert Vite or Next until confirmed.

4. component_architecture

The SPA is a layered composition. Top to bottom: **routing shell** → **auth/session context** → **feature screens** → **reusable UI primitives** → **the typed API client**. Business state lives in the server; the dashboard holds only session + view state (§7).

```
AppRoot
├── AuthProvider (Auth-Context-React - session,
tokens, refresh) [UNVERIFIED brick]
│   └── ApiClientProvider (typed fetch client; injects
bearer; 401→refresh)
│       └── Router
│           ├── PublicRoute /login → LoginScreen
│           └── ProtectedRoute (requires session)
│               └── AppShell (UI-Components-React - nav,
header, toasts) [UNVERIFIED]
│                   ├── / →
DashboardOverview (Dashboard-Analytics-React) [UNVERIFIED]
│                   ├── /artifacts/upload →
ArtifactUploadScreen
│                   ├── /releases, /releases/:id →
ReleaseList / ReleaseDetail
│                   ├── /deployments, /deployments/:id →
DeploymentList / DeploymentDetail
│                   └── /fleet, /fleet/:deviceId →
FleetHealth / DeviceDetail
└── (cross-cutting) ErrorBoundary, ToastHost, RoleGate
```

Component layers

Layer	Responsibility	Source
AuthProvider / session	Hold tokens, expose login/logout/refresh, gate routes.	Auth-Context-React (UNVERIFIED) wrapped by Helix glue.
ApiClientProvider	Single typed client; attaches Authorization: Bearer, Accept-Encoding: br, gzip; transparent 401→refresh→retry; maps the §6 error envelope to typed errors.	Helix-local (no auth/transport logic re-implemented; §5).
AppShell / primitives	Layout, nav, tables, forms, modals, toasts, badges, file-drop.	UI-Components-React (UNVERIFIED).
Analytics widgets	Fleet KPIs, deployment-progress charts, status distributions.	Dashboard-Analytics-React (UNVERIFIED).
Feature screens		Helix-local.

Layer	Responsibility	Source
	The four screen families (§9), each thin over the API client.	
RoleGate	Hide/disable actions above the session's RBAC role (§7.3).	Helix-local, fed by roles claim.

Decoupling (§11.4.28): UI primitives carry no API knowledge; the API client carries no view logic; screens orchestrate the two. This mirrors the server's seam discipline and keeps the brick boundaries clean.

5. catalogue_brick_reuse

Per catalogue-first (§11.4.74) the dashboard composes from the three `*-React` bricks named in master §10 / D7 — **but their catalogue membership and public API are UNVERIFIED** (`../..../00-master/submodule_reuse_map.md` §7 deliberately omits them from the verified catalogue §9). No bespoke equivalents are built where a brick covers the need; where a brick cannot be confirmed, §13.1 defines the fallback.

Concern	Brick	Class	Notes
Session / token context, login/logout/refresh, route guarding	Auth-Context-React	reuse (UNVERIFIED)	Wraps OAuth2/JWT client-side; Helix wires policies + the <code>/auth/*</code> calls (§7, §9.1). No token logic hand-rolled.
UI primitives — layout, nav, tables, forms, modals, toasts, file-drop	UI-Components-React	reuse (UNVERIFIED)	All screens compose these; no bespoke design system.
Analytics/ visualization — KPI tiles, charts, status distributions	Dashboard-Analytics-React	reuse (UNVERIFIED)	Powers the overview + fleet/telemetry views (§9.4).
Typed transport to <code>/api/v1</code> (bearer, Brotli accept, error mapping)	Helix-local ApiClientProvider	new (thin glue)	Not a brick concern; mirrors server transport contract (<code>endpoints.md</code> §3, §6). No auth/crypto re-implemented — tokens come from Auth-Context-React.

Upstream-addition candidates (extend, contributed back — never forked), each UNVERIFIED pending brick inspection: a multipart file-upload-with-progress primitive in UI-Components-React (for §9.2) and a transparent **refresh-rotation interceptor** in Auth-Context-React (for §7.2) if not already present.

6. route_map

All routes except /login are wrapped by ProtectedRoute (requires a valid session; §7). Routes whose underlying endpoint is **deferred** are listed but rendered behind a feature flag / “available in 1.0.1” placeholder so the IA is stable.

Path	Screen	Primary endpoint(s)	Min role	MVP?
/login	LoginScreen	POST /auth/login, POST /auth/refresh	none (public)	yes
/	DashboardOverview	GET /releases, GET /deployments/{id} (recent), GET /devices/{id}/status (sampled)	viewer	yes
/artifacts/upload	ArtifactUploadScreen	POST /artifacts/upload	operator	yes
/artifacts/:artifactId	ArtifactDetail	GET /artifacts/{artifactId}	viewer	yes
/releases	ReleaseList	GET /releases	viewer	yes
/releases/new	ReleaseCreate	POST /releases	operator	yes
/releases/:releaseId	ReleaseDetail	GET /releases/{releaseId}	viewer	yes
/deployments	DeploymentList	GET /deployments/{id} (per row)	viewer	yes
/deployments/new	DeploymentCreate	POST /deployments	operator	yes
/deployments/:deploymentId	DeploymentDetail	GET /deployments/{deploymentId}	viewer	yes
/fleet	FleetHealth	device list + GET /devices/{id}/status; telemetry overview (G4, PARTIAL)	viewer	partial
/fleet/:deviceId	DeviceDetail	GET /devices/{deviceId}/	viewer	partial

Path	Screen	Primary endpoint(s)	Min role	MVP?
		status; per-device telemetry history (G4, PARTIAL)		
/rollouts/*	(staged rollout)	staged-rollout API (1.0.1, G7)	—	deferred
/groups/*	(device-group CRUD)	group API (1.0.1, G5)	—	deferred
/audit	(audit log viewer)	audit API (1.0.1, G3)	—	deferred

Notes: there is no GET /releases-list-less fallback — list endpoints are paginated (?limit/?cursor, endpoints.md §2) and the tables consume next_cursor. A GET /deployments *list* endpoint is **not** defined in endpoints.md (only GET /deployments/{id}); the DeploymentList screen therefore composes its rows from the deployment ids surfaced by releases/overview, and **a deployments-list endpoint is flagged as a follow-up requirement** (see §13, server-side gap) — the dashboard does not invent it.

7. state_and_auth_context

7.1 state model

Two kinds of state, deliberately separated:

- **Session/auth state** (client-owned, in AuthProvider): access token (in memory), refresh token (see §10 for storage), decoded claims (sub, roles, exp), authentication status, and the current principal. This is the only long-lived client state.
- **Server state** (cache-owned, in the query layer): releases, deployments, devices, artifacts, telemetry — never duplicated into a global store; fetched, cached, and invalidated by the query cache keyed on the endpoint + params. Mutations (POST /artifacts/upload, /releases, /deployments) invalidate the relevant list caches on success.
- **Ephemeral view state** (component-local): form inputs, modal open/close, upload progress, pagination cursors.

There is **no** Redux-style global business store; the server is the single source of truth (matches the master decoupling stance).

7.2 auth context (JWT + refresh rotation)

The AuthProvider (over Auth-Context-React, UNVERIFIED) implements the OAuth2 flow from endpoints.md §4 / §7:

1. **Login** — POST /api/v1/auth/login with {username, password} → TokenResponse {access_token, token_type:"Bearer", expires_in

- (≈900s), refresh_token, roles}. The access JWT is held **in memory**; the refresh token is persisted per §10.
2. **Authenticated calls** — ApiClientProvider attaches Authorization: Bearer <access> to every /api/v1 request and Accept-Encoding: br, gzip (endpoints.md §3).
 3. **Silent renewal with rotation** — on a 401 UNAUTHENTICATED (or proactively before exp), call POST /api/v1/auth/refresh with the stored refresh token → a **new** access+refresh pair; the **old refresh token is invalidated server-side (rotation, endpoints.md §7.2)** and the stored one is overwritten. The original request is retried once. A single in-flight refresh is shared across concurrent 401s (no refresh stampede).
 4. **Failed refresh** — if refresh returns 401 (expired/revoked/already-rotated), the session is cleared and the user is redirected to /login with a “session expired” notice.
 5. **Logout** — clear in-memory access token and stored refresh token; redirect to /login. (Server-side refresh revocation endpoint is not defined in endpoints.md MVP; client-side clear is the MVP behavior — flagged in §13.)

7.3 rbac in the ui

The roles claim drives a RoleGate that hides/disables actions the principal cannot perform, mirroring the server route policy (endpoints.md §4.2):

Role	Dashboard capability
admin	Everything below, plus (future) user/audit admin.
operator	Upload artifacts, create releases, create deployments; all reads.
viewer	Read-only: lists + detail for releases, deployments, devices, telemetry.
device	N/A — device tokens never reach the dashboard; the SPA is for human roles.

The RoleGate is **UX only**; the server enforces RBAC authoritatively and returns 403 FORBIDDEN regardless of what the UI shows.

8. api_client_and_screen_endpoint_mapping

A single typed ApiClientProvider is the only thing that talks to /api/v1. It: attaches the bearer token and Accept-Encoding: br, gzip; sends/receives application/json (except the multipart upload, §9.2); echoes X-Request-Id into error toasts for correlation; and maps the server error envelope (endpoints.md §6: {error:{code,message,request_id,details[]}}) into typed client errors so each screen can render field-level details and the stable code.

Screen → **endpoint matrix** (every screen traces to documented endpoints):

Screen	Action	Endpoint (endpoints.md §)	Success
LoginScreen	submit credentials	POST /auth/login (§7.1)	200 TokenResponse
(any)	silent renew	POST /auth/refresh (§7.2)	200 TokenResponse
ArtifactUploadScreen	upload + validate	POST /artifacts/upload (§9.1)	201 Artifact
ArtifactDetail	read metadata	GET /artifacts/{id} (§9.2)	200 Artifact
ReleaseCreate	publish release	POST /releases (§10.1)	201 Release
ReleaseList	list	GET /releases (§10.2)	200 ReleaseList
ReleaseDetail	read	GET /releases/{id} (§10.2)	200 Release
DeploymentCreate	deploy (all-targets)	POST /deployments (§11.1)	201 Deployment
DeploymentDetail	progress	GET /deployments/{id} (§11.2)	200 DeploymentStatus (+ {pending, downloading, installed, success})
FleetHealth / DeviceDetail	device status	GET /devices/{deviceId}/status (§8.2)	200 DeviceStatus
DeviceRegister (admin)	provision device	POST /devices/register (§8.1)	201 DeviceRegistered
FleetHealth (telemetry)	aggregates / history	telemetry read API (G4 PARTIAL)	—

Health endpoint: the SPA may surface a server-health badge from the platform health endpoint if exposed; endpoints.md does not define a public /health JSON route in MVP, so the badge is **best-effort and flagged UNVERIFIED** (§13) rather than asserted.

9. screen_specifications

9.1 login_screen

Goal: authenticate an operator and establish a session.

- **Layout:** centered card (from UI-Components-React, UNVERIFIED) — username (email), password, submit; inline error region; disabled submit while in-flight.
- **Validation (client, UX only):** non-empty username + password before enabling submit.
- **Flow:** submit → POST /api/v1/auth/login (§7.1). On 200, store tokens (§7.2 / §10) and redirect to the originally requested protected route (or /). On 401 UNAUTHENTICATED, show “invalid credentials” without revealing which field. On 429 RATE_LIMITED, show a “too many attempts, retry after N seconds” message sourced from Retry-After (login is strictly rate-limited server-side, endpoints.md §5).
- **Security:** password is never logged or placed in URL/query; submitted over TLS; the field uses type=password + autocomplete=current-password.

9.2 artifact_upload_screen

Goal: upload an OTA artifact and surface the server-side validation result, step by step, as the **S1–S6 validation-feedback chain**.

- **Inputs:** a file drop-zone (.zip / payload.bin) + a metadata form binding the ArtifactUploadMetadata fields (endpoints.md §9.1): sha256 (lowercase hex), signature (base64 detached), version, os, target_model, and the AOSP streaming fields file_hash/file_size/metadata_hash/metadata_size/payload_offset/payload_size. The operator MAY paste a precomputed sha256; the client MAY also compute it in-browser for an early local mismatch warning (server remains authoritative).
- **Request:** POST /api/v1/artifacts/upload as multipart/form-data with parts file (binary, ZIP_STORED) and metadata (JSON), with an upload **progress bar** (extend candidate on UI-Components-React, §5).

S1–S6 validation-feedback chain (UI states mapped onto the server pipeline + endpoints.md §9.1 checks/codes; the dashboard renders each as a step that turns green/red as the single server response resolves — the server returns one terminal result, so S1–S4 client states are *pre-flight/progress* and S5/S6 reflect the server verdict):

Step	Meaning	Surfaced from
S1 structure / media type	File selected, type/size sane before send.	client pre-check; server 415 UNSUPPORTED_MEDIA_TYPE / 400 VALIDATION_FAILED
S2 size within cap	Under the configured size limit.	client pre-check; server 413 PAYLOAD_TOO_LARGE
S3 metadata complete	All required metadata fields present + well-formed.	client schema validation; server 400

Step	Meaning	Surfaced from
		VALIDATION_FAILED with details[]
S4 hash match	Computed SHA-256 of bytes == declared sha256.	server 422 HASH_MISMATCH (verified server-side; client may pre-warn)
S5 signature valid	Detached signature verifies against the trusted key.	server 422 SIGNATURE_INVALID
S6 stored / verified	Artifact persisted (ZIP_STORED), verified=true.	server 201 Artifact {artifact_id, verified:true, storage_ref, ...}

Ordering note (gap G2): endpoints.md §9.1 lists hash (S4) before signature (S5); the spec carries the **hash-before-signature** order, but the exact S2-before-S3 vs S4-before-S5 ordering inside ota-artifact-validator is **UNVERIFIED** at brick level (G2 / [../security/signing_verification.md](#)). The UI **MUST** render whatever terminal error.code the server returns and **MUST NOT** assert an ordering the server didn't confirm.

- **Success:** on 201, show the Artifact summary and a primary CTA “Create release from this artifact” → prefilled ReleaseCreate (§9.3).
- **Errors:** each 4xx maps to the corresponding S-step turning red with the server message + details[]; the operator can fix metadata and resubmit without re-selecting the file.

9.3 release_and_deployment_management

Releases

- **ReleaseList** — paginated table (GET /releases, filters ?os/?target_model/?status); columns: version, os, target_model, status, created_at; row → ReleaseDetail.
- **ReleaseCreate** — form binding ReleaseCreate (endpoints.md §10.1): artifact_id (prefilled from §9.2), version, os, target_model, notes, min_current_version. Submit → POST /releases. Surface 409 VERSION_NOT_MONOTONIC prominently (“version must be strictly greater than the latest release for this os + target”) and 404 if the artifact is unknown/unverified.
- **ReleaseDetail** — read-only Release; primary CTA “Deploy this release” → DeploymentCreate prefilled.

Deployments (all-targets for MVP)

- **DeploymentCreate** — form binding DeploymentCreate (endpoints.md §11.1): release_id (prefilled), strategy (**locked to all-targets** in MVP; the control is a disabled/single-value selector with an inline “staged rollout arrives in 1.0.1” note — any other value yields 400), optional group. Submit → POST /deployments; show 409 CONFLICT if an active deployment already targets the set.

- **DeploymentDetail** — GET /deployments/{id} → DeploymentStatus with the aggregate progress {pending, downloading, installed, succeeded, failed} rendered as a progress bar + counts (chart via Dashboard-Analytics-React, UNVERIFIED). Auto-refresh on the query-cache poll interval. **No pause/resume/abort controls in MVP** (those belong to the staged-rollout lifecycle, G7/1.0.1); the layout reserves the action row.

9.4 fleet_health_and_telemetry_views

Goal: give the operator fleet-wide health and per-device drill-down.

- **FleetHealth** — a device list with health badges and KPI tiles (online/last-seen, current-version distribution, in-progress vs failed counts). Per-row/per-device status comes from GET /api/v1/devices/{deviceId}/status (endpoints.md §8.2) returning DeviceStatus {current_version, target_version, last_seen, update_state, active_slot, health}. update_state is the 7-value UpdateState enum (idle,download_started, installing,installed,verifying,success,failure).
- **DeviceDetail** — full DeviceStatus + (when G4 lands) a telemetry event history for the device. Telemetry events are the 6-value TelemetryEventType (download_started,installing,installed,verifying,success,failure — **no idle**, endpoints.md §12.2).
- **Telemetry read API is PARTIAL (gap G4):** ingest (POST /client/telemetry) exists, but read/aggregation endpoints (per-device history, /telemetry/overview) are **not yet specified** (additions_synthesis.md §8 G4). The dashboard therefore:
 1. derives what it can from DeviceStatus + DeploymentStatus aggregates (both MVP), and
 2. renders telemetry-history panels in a **graceful empty**/"available when the telemetry read API ships" state rather than calling an undefined endpoint. The dashboard does **not** invent telemetry-read routes; it binds them when G4 specifies them.

10. security_considerations

- **Token storage:** the access JWT lives in memory only (cleared on reload; re-minted via refresh). The refresh token storage trades off XSS vs CSRF: the preferred MVP option is a **secure, HttpOnly, SameSite=Strict cookie set by the server** so JS cannot read it (mitigates XSS exfiltration) — **UNVERIFIED** whether the MVP auth flow / auth brick issues the refresh token as a cookie vs a JSON body field (endpoints.md §7.1 shows it in the JSON body). If it is JSON-only, the fallback is in-memory + re-login on reload, **never** localStorage for the refresh token. This choice is flagged for confirmation in §13.
- **Refresh rotation** (§7.2) limits the blast radius of a leaked refresh token: each use invalidates the prior token server-side.
- **Transport:** TLS 1.3 only; HTTP/3→HTTP/2 negotiated by the server (endpoints.md §3); the SPA sends Accept-Encoding: br, gzip and never downgrades.
- **No secrets in the bundle:** signing keys, server secrets, and the build-pipeline private key are **never** present in the SPA. The dashboard only ever holds the operator's short-lived tokens.

- **RBAC is server-authoritative** (§7.3); the UI’s RoleGate is defense-in-UX only.
- **Content Security Policy:** restrict connect-src to the API origin, disallow inline script, set frame-ancestors 'none'; X-Content-Type-Options nosniff. (Concrete CSP header values are a deployment concern — flagged, not asserted here.)
- **Error hygiene:** the SPA renders the safe server message + code + request_id (endpoints.md §6) and never surfaces stack traces; it logs no tokens.

11. new_public_repository_plan

Per locked decision **D4** (master §2/§10: “New reusable submodules get PUBLIC repos auto-created on GitHub + GitLab”), the dashboard ships as a **NEW public repository** (additions_synthesis.md §9.1 routes G6 to “new public repo”).

Field	Plan
Repo name	ota-dash-board (aligns with the ota-* NEW-submodule naming in master §10).
Visibility	PUBLIC (D4).
Hosts	GitHub + GitLab (D4), mirrored to GitFlic + GitVerse per the multi-upstream push convention (additions_synthesis.md §4; master §10 “tag mirroring”).
Org / owner	Same org pattern as the OTA submodules (the HelixDevelopment/OTA org used for helix_ota; the vasic-digital org is for catalogue bricks). UNVERIFIED which exact org owns the dashboard repo — confirm before creation.
Boundary	UI only — consumes /api/v1; holds no OTA business logic; depends on ota-protocol (TypeScript wire types) + the three *-React bricks. Decoupled, independently testable (§11.4.28).
Contents	the React SPA, the typed API client, the four screen families, CI (build + four-layer tests + bundle/secret guard), container build on the containers substrate.
Pre-creation gate	Listed here in the MVP spec before repo creation (master §10 “final list confirmed in the MVP spec immediately before creation”).
Multi-upstream	submodule-commit-first + tag mirroring across all four hosts (master §13 governance row).

The three ***-React** bricks are **consumed**, not created here; if any does not exist as a real reusable repo, §13.1 governs the fallback (it does not become a silently-invented dependency).

12. testing_four_layer

Per master §13 / Constitution §1 four-layer testing with no-bluff positive evidence (§7.1). Safety-relevant UI paths here are **the upload S1–S6 feedback** (§9.2) and **the auth/refresh-rotation flow** (§7.2).

1. **source-presence gate**. Static assertion that the code artifacts implementing this spec exist: every route in §6 has a screen component; every screen's API call targets a documented endpoints.md route; the typed client's request DTOs match `ota-protocol / openapi.yaml`. Fails if a screen calls an undefined endpoint or a route has no component.
2. **artifact gate (bundle shipped)**. Build the SPA and assert the *built* bundle: it contains the four screen families, contains **no** secret/private-key material (secret-scan guard), and pins `connect-src` to the API origin in the emitted CSP. Boot the SPA against a mock `/api/v1` and assert the route table renders.
3. **runtime / integration**. Component + end-to-end tests against a mocked (and, in the dev stack, real containerized) control plane: `login` → `200 tokens` → `upload valid artifact` → `S6 verified:true` → `create release` → `create all-targets deployment` → `deployment progress renders` → `device status renders`. Negatives: bad credentials → `401 message`; tampered artifact upload → **S5 SIGNATURE_INVALID** rendered red; non-monotonic release → `409 surfaced`; a `401` mid-session triggers **refresh-rotation** and a single transparent retry; failed refresh → redirect to `/login`; a viewer session has operator actions gated.
4. **mutation meta-test (PASS→FAIL on negation)**. Mutate the UI and assert the suite flips: drop the bearer header in the client (auth integration tests must fail); make the upload screen ignore `422 SIGNATURE_INVALID` and show success (S5 negative test must fail); disable refresh rotation so the old token is reused (rotation test must fail); remove the `RoleGate` (the viewer-gating test must fail). A mutation that breaks nothing exposes a coverage hole.

13. anti_bluff_unverified_register

Per HelixConstitution §7.1 / §11.4.6 (anti-bluff / no-guessing), the following **MUST NOT** be propagated as fact:

- **The three *-React bricks** (UI-Components-React, Dashboard-Analytics-React, Auth-Context-React) are named in master §10 / D7 but are **NOT on the verified catalogue** (`../00-master/submodule_reuse_map.md` §7 explicitly omits them). Their existence, ownership, repos, and public APIs are **UNVERIFIED**. Every “reuse” of them in §4/§5 is conditional on inspection.
- **Toolchain** (Vite vs Next; bundler; component-library internals) is **UNVERIFIED**; deferred to brick inspection (§3.2). Do not assert a build tool.
- **Refresh-token transport** (cookie vs JSON body) and a server-side **refresh-revocation/ logout** endpoint are **UNVERIFIED** against the MVP auth flow (endpoints.md §7 shows the refresh token in the JSON body and defines no logout route). §10 picks the safer cookie option but flags it.

- **GET /deployments list endpoint** and a public **/health JSON endpoint** are **not defined** in `endpoints.md` MVP; the DeploymentList screen and the health badge are noted as **server-side follow-ups**, not invented client routes (§6, §8).
- **Telemetry read/aggregation API (gap G4)** is **PARTIAL/undefined**; the fleet/telemetry views degrade gracefully and bind it only when G4 specifies it (§9.4).
- **ota-artifact-validator S2-before-S3 / hash-before-signature ordering (gap G2)** is **UNVERIFIED** at brick level; the UI renders the server's terminal `error.code` and asserts no ordering (§9.2).
- **The dashboard repo's owning org** is **UNVERIFIED** — confirm before creation (§11).
- **HelixConstitution clause numbers** (§11.4.6, §11.4.4, §11.4.61, §11.4.74, §11.4.28, §7.1) are carried from corpus convention and are **UNVERIFIED** against the authoritative Constitution text.

13.1 fallback if a *-React brick is unconfirmed

If, on inspection, a *-React brick does not exist or does not cover the need, the dashboard substitutes a **single, well-established, verified third-party library** for that concern (a component library for UI-Components-React; a charting library for Dashboard-Analytics-React; an OIDC/JWT client-context for Auth-Context-React), recorded as a corrected binding here and in the reuse map — **never** a silently-invented dependency, and only after the catalogue-first check fails (§11.4.74).

14. compliance_notes_helixconstitution

Clause numbers are carried from corpus convention and are **UNVERIFIED** against the authoritative Constitution text (the Constitution file is not present in this repository).

Clause (label)	How this spec complies
§11.4.61 (ToC mandatory)	Metadata table first, ToC immediately after, numbered + anchored.
§7.1 / §11.4.6 (anti-bluff / no-guessing)	Every brick, endpoint, and toolchain claim traces to <code>endpoints.md</code> / <code>master</code> / <code>reuse-map</code> or is tagged UNVERIFIED (§13); the unconfirmed *-React bricks, refresh-token transport, deferred endpoints, and clause numbers are never asserted as fact.
§11.4.74 (catalogue-first reuse)	Composes from the *-React bricks where they can be confirmed; a fallback (§13.1) only after the catalogue-first check fails; no auth/transport logic hand-rolled (tokens from the auth brick, contract from <code>ota-protocol/openapi.yaml</code>).
§11.4.28 (decoupling)	UI primitives carry no API knowledge; the API client carries no view logic; the

Clause (label)	How this spec complies
§1 / §1.1 (four-layer + mutation)	dashboard holds no OTA business logic (§4). §12 defines source-presence → artifact/bundle → runtime/integration → mutation meta-test; safety focus on upload S1–S6 + refresh rotation.
D4 (PUBLIC new repos)	ota-dashboard planned PUBLIC on GitHub + GitLab, mirrored to GitFlic/GitVerse, listed before creation (§11).
D7 (React dashboard)	React SPA over /api/v1; login + upload + deploy + fleet/telemetry (§9).
LOCKED stack honored	Consumes the Go + Gin + Brotli + HTTP/3→HTTP/2 REST surface (Accept-Encoding: br, gzip; §3, §8); REST primary; no gRPC.
LOCKED scope honored	All-targets deployments only (staged rollout deferred to 1.0.1, G7); no rollback/group/audit UI in MVP — reserved, not built (§1, §6).